

CSCE 605 COMPILER DESIGN · MODULE 12 · LECTURE NOTES

Runtimes: Garbage Collection, JIT, and Scripting

What runs alongside the code you generated.

MODULE OUTCOMES

- Compare garbage collection algorithms on their real trade-offs.
- Explain generational collection and the generational hypothesis.
- Explain JIT tiering, deoptimisation, and inline caches.
- Design the embedding of a scripting language in an engine.
- Explain why GC pauses and frame budgets conflict.

1 Garbage collection

Algorithm	Mechanism	Trade
Reference counting	Each object counts its references; freed when the count reaches zero.	Naturally incremental with no pauses, and cycles leak unless a separate cycle collector runs. Every pointer assignment costs an increment and a decrement, which is a continuous throughput tax.
Mark-sweep	Trace from the roots marking reachable objects, then free everything unmarked.	Handles cycles correctly. Fragments the heap over time, and the trace is a pause.
Mark-compact	Mark, then slide the survivors together to one end.	No fragmentation , at the cost of moving objects — which means every pointer to them must be updated, so native code holding raw pointers must be accounted for.
Copying (semispace)	Copy live objects into a fresh space; the old space is then entirely free.	Allocation becomes a pointer bump (see below), and it costs twice the memory since half is always empty.
Generational	Segregate by age and collect young objects frequently.	The single largest practical win , and it requires a write barrier — see the callout.
Concurrent / incremental	Collect alongside the running program, in small steps or on another thread.	Much shorter pauses , somewhat lower total throughput , and considerably more implementation complexity.

THE GENERATIONAL HYPOTHESIS IS WHY GC IS AFFORDABLE

Most objects die young. This is an empirical observation that holds across almost every program in almost every language — temporaries, intermediate results, short-lived closures, and iteration state vastly outnumber long-lived data.

So collect the young generation frequently and cheaply. The key property is that a **copying collector's cost is proportional to the survivors, not to the garbage** — dead objects are never touched at all.

If 95% of the young generation is dead, a collection copies 5% and resets a pointer. It is close to free, and it can therefore be run often enough that the young generation stays small and cache-resident.

The cost is a write barrier. Because a young object may be referenced only from an old one, every pointer store must be checked and recorded if it creates an old-to-young reference, so the young collection knows its full root set without scanning the old generation. **A small tax on every pointer write, paid to make collection cheap** — and it is a trade that has won decisively, which is why essentially every production collector is generational.

ALLOCATION IN A COPYING COLLECTOR IS A POINTER BUMP

Allocation is: add the object size to a pointer, compare against the limit, return the old pointer value.

Three instructions, no free list, no size classes, no search, and no metadata per object.

That is genuinely faster than malloc, which must find a suitable free block, maintain its own bookkeeping, and handle fragmentation. **'GC is slow' is too simple a claim**, and allocation-heavy code can be faster under a good collector than under manual management.

And survivors are copied into contiguous memory, so objects that survive together end up adjacent — which improves locality (CSCE 614) in a way manual allocation does not naturally provide.

The real costs are pause time and memory. A garbage collected program typically needs two to five times its live-set size to perform well, and performance degrades sharply below that because collections become frequent and survivor rates rise. **Pauses and memory are the honest trades**, not throughput — and pauses are what makes this a problem for engines (§3).

2 JIT compilation

TIERING: START FAST, THEN GET FAST

Tier 0 interprets. Zero compilation cost, slow execution. **Most code runs here and never leaves**, which is the point — compiling code that runs twice is a waste.

Tier 1 compiles quickly with minimal optimisation, while instrumenting the code to gather a profile: which types appear at each site, which branches are taken, which call targets occur.

Tier 2 compiles the hot code hard, using that profile to specialise aggressively — assuming the observed types, inlining through the observed call targets, laying out the observed hot path straight.

And deoptimises when an assumption breaks. The specialised code guards every assumption it made; when a guard fails it must transfer control back to the interpreter **mid-function, reconstructing the interpreter's entire state — locals, operand stack, program counter — from the optimised frame**, in which several of those values may be in registers, folded into constants, or eliminated entirely. **That reconstruction is the hard part of building a JIT**, and it is what the optimiser must preserve enough information to make possible — the same problem as Module 10 §4's debug information, with correctness rather than convenience at stake.

INLINE CACHES: SPECULATING ON OBJECT SHAPE

In a dynamic language, `x.foo` requires a lookup — the object's layout is not known statically, so the field's offset must be found at runtime, potentially through a prototype chain.

But at any given call site the object's shape is usually the same every single time. The loop that processes a thousand particles sees a thousand objects with identical layout.

So cache the lookup's result at the site, keyed on the shape. A **monomorphic** site — one shape ever observed — becomes a shape check and a load at a fixed offset, **nearly as fast as a C struct field access**.

Polymorphic caches hold a handful of shapes and check each; megamorphic sites give up and fall back to the full lookup. This single technique is most of the reason JavaScript is fast, and it is the direct reason that keeping object shapes consistent — initialising all fields in the same order, not adding properties later — is standard JavaScript performance advice. The advice is about keeping sites monomorphic.

3 Scripting in an engine

Option	Character	Cost
Lua or LuaJIT	Tiny, fast, and designed from the start to be embedded. LuaJIT's tracing JIT is remarkably good.	Small standard library , and LuaJIT's language version is frozen at 5.1.
C# via Mono or IL2CPP	A full language with real tooling, debuggers, and a large ecosystem.	A large runtime to ship and GC pauses to manage — which is Unity's situation and the source of most of its performance guidance.
Visual scripting	Non-programmers can author behaviour , which can be transformative for a content team.	Diffing and merging are painful — graphs do not merge — and complex logic becomes unreadable. Version control pain is the usual complaint.
WebAssembly	Any source language, sandboxed, with predictable performance.	Host interop is awkward; passing structured data across the boundary requires marshalling.
A custom DSL	Exactly fits your domain, and can be made safe and fast by construction.	You own the tooling forever — editor support, debugger, error messages, documentation. Usually underestimated by a large factor.
Native code with hot reload	Full speed, no boundary.	A script bug crashes the engine , and reload has to handle state migration.

The deciding question is **who writes the scripts**. Designers, gameplay programmers, and external modders want substantially different things from this list, and **the answer is a people question rather than a technical one** — which is worth recognising before the benchmark comparison starts.

GC PAUSES AND FRAME BUDGETS ARE IN DIRECT CONFLICT

A 60 Hz frame budget is 16.7 milliseconds, and most of it is already committed to simulation, animation, and rendering. A 10 ms GC pause is a visible hitch; a 50 ms pause is a stutter every player notices.

So engines fight the allocator rather than tuning the collector: pool objects, reuse buffers, preallocate at load time, and avoid allocating anything at all in the per-frame path.

Unity's performance guidance is essentially 'do not allocate during gameplay', which is a strong signal about where the problem actually lies — the recommended solution to having a garbage collector is to not generate garbage.

Incremental and concurrent collectors trade total throughput for shorter pauses, which is exactly the right trade here. A frame budget cares about the worst case, not the mean — the same argument as CSCE 650 Module 06 made about VR latency, and CSCE 735 Module 07 made about reporting distributions rather than averages. **A collector that is 10% slower overall and never pauses for more than 1 ms is strictly better for an interactive application**, and that is not a judgement a throughput benchmark can make.

4 The interface

- **Object lifetime across the boundary.** A script holds a reference to a native object; the engine destroys it; the script uses it. **Handles with generation counters** rather than raw pointers are the standard answer, so a stale reference is detectable rather than catastrophic.
- **Two collectors, or one collector and manual memory, that cannot see each other's references.** A cycle spanning the boundary — a native object holding a script object holding the native object — **leaks permanently**, because neither side can prove the other's half unreachable.
- **Error propagation.** A script error must not take the engine down, must be caught at a defined boundary, and must still produce a useful message with a script-level stack trace — which means the embedding must map runtime errors back to script source locations (Module 10 §4 again).
- **Threading.** Most script runtimes are single-threaded by design, while the engine is thoroughly parallel (CSCE 735). **Deciding where scripts may run, and what they may touch, is a design decision that cannot be deferred.**
- **And call overhead.** Crossing the boundary has a real cost — argument marshallng, stack setup, possibly a lock — and **calling into script once per entity per frame is a measurable fraction of the budget. Batch it:** one call processing a thousand entities beats a thousand calls. **Design the boundary first**, because it is much harder to narrow afterwards and it determines everything else.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Jones, Hosking & Moss — The Garbage Collection Handbook	The reference for §1. Library copy; the authors maintain a free bibliography and the first chapters set up the trade-offs well.
Nystrom — Crafting Interpreters, 'Garbage Collection' (free)	A complete mark-sweep collector implemented and explained, including the root-finding problems that make it tricky.
V8 and SpiderMonkey engineering blogs (free)	Tiering, inline caches, and deoptimisation described by the people implementing them, with measurements. The best free source for §2.
Roberto Ierusalimschy et al. — The Implementation of Lua 5.0 (free)	How a language designed for embedding is built — the §3 option, from its authors. Short and unusually clear.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement mark-sweep collection for your interpreter.
- 2 **Add a copying young generation** with a write barrier and measure allocation throughput before and after.
- 3 **Measure the generational hypothesis directly**: instrument object lifetimes and plot the survival curve.
- 4 Compare allocation cost against malloc for many small objects.
- 5 Measure your collector's pause times and plot the distribution, not the mean.
- 6 **Vary the heap size from 1× to 5× the live set** and plot throughput. Explain the shape.
- 7 Implement a monomorphic inline cache for field access and measure it.
- 8 **Make a call site polymorphic** and then megamorphic, and measure the degradation at each step.
- 9 Embed Lua in a small program and expose a native type with handles and generation counters.
- 10 **Measure the boundary crossing cost** and find the batch size at which it stops mattering.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name six collection algorithms and their trades.
- 2 State the generational hypothesis and explain why it makes GC cheap.
- 3 What is a write barrier and why is it needed?
- 4 Why is allocation in a copying collector fast, and what are the real costs?
- 5 Describe JIT tiering and say what deoptimisation must reconstruct.
- 6 What is an inline cache, and what are monomorphic and megamorphic sites?
- 7 Compare six scripting options and say what decides between them.
- 8 Why do GC pauses conflict with frame budgets, and what do engines do?
- 9 Name five things that make an embedding painful.